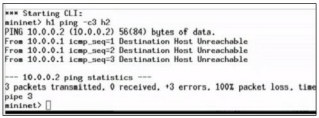

Figure 3: Remote controller without any entries in switch and thus dropping all the packets

```
remote
```

After successfully starting, it will show the message that switches have been connected with the *{MAC address}*. To verify the connections established by default, use the following command:

```
pingall
```

Here, all the hosts will be unreachable to one another. This is because we have set the remote controller, but have not initiated it or, in other words, there is no entry in the controller for how to handle incoming packets at the switches. Thus, when OF-enabled switches ask for the decision from the controller, it simply gives the command to drop the packets. Figure 3 shows a snapshot of the following command:

```
h1 ping -c3 h2
```

We can now control each and every host of the network in the emulator, virtually. For that, we can use the following command:

```
xterm h1 h2 h3
```

Three small terminal-like windows will pop up. To check that everything is working fine, we can just capture the packages. Assuming that support for Mininet in Wireshark is also installed, run the following commands:

```
tcpdump -XX -n -I h1-eth0
tcpdump -XX -n -I h2-eth0
```

Now, we are monitoring the traffic on *h1* and *h2* hosts. Let's run the ping command in *xterm* of *h3*.

```
ping -c1 10.0.0.1
```

After running this command, we can see the ARP requests in the *h1* terminal.

Customising an existing POX controller to act as a firewall and load balancer

Till now, we were making the controller work like a hub. To make it work like a switch or learning switch, we can make changes in the file *of_tutorial*, and replace the statement *act_like_hub()* to *act_like_switch()*.

An SDN based firewall

For a firewall kind of application, we need to have a mediator that can filter out the packets based on some conditions. Let us consider an example with MAC address filtration. For that we require to build a look-up table that can hold the MAC addresses of all the devices allowed to communicate or transfer data.

To make the controller communicate with switches, the former needs to send the messages to the latter. As and when a connection initiates a switch, even *ConnectionUp* is fired. Thus, tutorial code creates a *Connection* object, which can later be used to send the messages to the switches using the function *connection.send()*. At that time, we can decide the default flow using the *ofp_action_output* function, as shown below:

```
out_action = of.ofp_action_output (port = of.OFPP_FLOOD)
```

Here, *OFPP_FLOOD* even decides on flooding, which means that the packets will be forwarded to every port except the one on which the packet had arrived.

A firewall needs to restrict the packets on the basis of several other criteria. For that, we need to create the object *ofp_match* class. Some important fields of this class are *dl_src*, *dl_dst* and *in_port*. Here, *dl* represents the data link layer, which means *src* and *dst* are MAC addresses of the source and the destination, respectively. To send the packet, we need to send a message using *ofp_packet_out*. The *send_packet()* method can be used in *of_tutorial*.

Here is an example:

```
def send_packet (self, buffer_id, raw_data, out_port, in_
port):
            """Sends the packet out of the specified port.

            If buffer_id is a valid buffer on the switch, use it;
otherwise, send the raw data in raw_data.
            The in_port is the port number that the packet arrived on.
Use OFPP_NONE if the packet is generated by you.

            """
            msg = of.ofp_packet_out()
            msg.in_port = in_port
            if buffer_id != -1 and buffer_id is not None:
                        msg.buffer_id = buffer_id
            else:
                        if raw_data is None:
                                    return
                        msg.raw_data = raw_data
      action = of.ofp_action_output (port = out_port)
            msg.actions.append (action)
            self.connection.send(msg)
```